

EXAMEN DE PROGRAMACIÓN – 1º GRADO SUPERIOR

Examen 4 · Herencia, polimorfismo, interfaces y clases abstractas

Herencia (extends), super, override

Polimorfismo y enlace dinámico

Clases abstractas y métodos abstractos

Interfaces (implements)

Modificadores final y static en herencia

Uso de ArrayList con tipos polimórficos

Asignatura	Programación (PRG)
Curso	1º Grado Superior — Desarrollo de Aplicaciones Web (DAW)
Duración	2 horas
Puntuación total	16 puntos
Convocatoria	Evaluación 2
Valoración	Herencia y polimorfismo

Parte 1: Opción múltiple y Verdadero/Falso

5 puntos — 0,5 ca

Para hacer que la clase `Perro` herede de `Animal` se usa:

- a) class Perro implements Animal
- b) class Perro extends Animal
- c) class Perro inherits Animal
- d) class Perro : Animal

RESPUESTA / SOLUCIÓN

b) class Perro extends Animal — extends indica herencia de clase; implements se usa con interfaces.

¿Qué palabra clave se usa para llamar al constructor de la clase padre dentro de un constructor hijo?

- a) this()
- b) base()
- c) super()
- d) parent()

RESPUESTA / SOLUCIÓN

c) super() — invoca el constructor del padre. this() invoca un constructor de la propia clase.

Una clase abstracta puede contener métodos concretos (con implementación).

RESPUESTA / SOLUCIÓN

Verdadero — Una clase abstracta puede mezclar métodos abstractos y métodos concretos.

Dado:

```
class Animal { void sonido() { System.out.println("genérico"); } }  
class Perro extends Animal { void sonido() { System.out.println("guau"); } }  
Animal a = new Perro();  
a.sonido();
```

- a) genérico
- b) guau
- c) Error de compilación
- d) Nada

RESPUESTA / SOLUCIÓN

b) guau — El enlace dinámico (polimorfismo) invoca la implementación de la clase real del objeto (Perro) aunque la referencia sea de tipo Animal.

Una interfaz en Java:

- a) Solo puede tener métodos abstractos (hasta Java 8 también default/static)
- b) Puede ser instanciada con new
- c) Solo puede heredar de una interfaz
- d) No puede tener constantes

RESPUESTA / SOLUCIÓN

a) Una interfaz declara métodos abstractos (y desde Java 8 métodos default/static) que las

clases que la implementan deben completar. No se puede instanciar con new.

Para que una clase implemente una interfaz se usa la palabra clave:

- a) extends
- b) uses
- c) implements
- d) inherits

RESPUESTA / SOLUCIÓN

c) implements — Una clase implementa una interfaz; puede implementar varias separadas por comas.

¿Cuál es el propósito de `@Override`?

- a) Crear un método nuevo
- b) Indicar que se sobrescribe un método heredado
- c) Hacer el método estático
- d) Hacer el método público

RESPUESTA / SOLUCIÓN

b) `@Override` es una anotación que indica que el método sobrescribe uno de la clase padre; ayuda al compilador a detectar errores de firma.

En Java una clase puede heredar de varias clases simultáneamente (herencia múltiple).

RESPUESTA / SOLUCIÓN

Falso — Java no permite herencia múltiple de clases. Sí permite implementar varias interfaces.

¿Qué elemento NO se puede heredar en una subclase?

- a) Métodos public
- b) Atributos protected
- c) Constructores
- d) Métodos protected

RESPUESTA / SOLUCIÓN

c) Los constructores no se heredan; el hijo los invoca mediante `super()`, pero no son miembros heredados.

Dado un método `abstract`, la clase que lo contiene debe ser:

- a) final
- b) abstract
- c) static
- d) Una interfaz o clase abstracta

RESPUESTA / SOLUCIÓN

b) abstract — Un método abstract declarado obliga a que su clase sea abstracta o sea una interfaz.

Parte 2: Análisis y depuración

3 puntos

El siguiente código tiene errores de herencia. Encuéntralos y corrige el diseño. (1,5 ptos)

```
abstract class Vehiculo {
    abstract void mover();
}
class Coche extends Vehiculo { }
class Moto extends Vehiculo {
    void mover() { System.out.println("La moto avanza"); }
}
public class Main { public static void main(String[] a) {
    Vehiculo v = new Coche(); // ¿compila?
    v.mover();
} }
```

RESPUESTA / SOLUCIÓN

1. La clase abstracta Vehiculo tiene un método abstracto `mover()`. La clase Coche no lo implementa y NO es abstracta, por lo que NO compila (debe implementar mover() o declararse abstract).
2. Corregir: `class Coche extends Vehiculo { void mover() { System.out.println("El coche avanza"); } }`.
3. Moto implementa mover() correctamente; con la corrección la salida sería "El coche avanza".

Determina la salida de este programa. (1,5 ptos)

```
abstract class Figura {
    abstract double area();
    void mostrar() { System.out.println("Área: " + area()); }
}
class Circulo extends Figura {
    private double radio;
    Circulo(double radio) { this.radio = radio; }
    double area() { return Math.PI * radio * radio; }
}
class Cuadrado extends Figura {
    private double lado;
    Cuadrado(double lado) { this.lado = lado; }
    double area() { return lado * lado; }
}
public class Main { public static void main(String[] a) {
    Figura[] f = { new Circulo(2), new Cuadrado(3) };
    for (Figura x : f) x.mostrar();
} }
```

RESPUESTA / SOLUCIÓN

Polimorfismo: cada Figura invoca su propia implementación de area().

Circulo de radio 2 → área = $\pi \cdot 4 \approx 12,566$.

Cuadrado de lado 3 → área = 9.

Salida: `Área: 12.566370614359172` y `Área: 9.0`.

Parte 3: Ejercicios de programación

7 puntos

Crea una jerarquía de clases para `Empleado`, `Directivo` (hereda) y `Tecnico` (hereda). Empleado tiene nombre y salario. Directivo añade `departamento`; Técnico añade `especialidad`. Sobrescribe `toString` en las tres y demuestra el polimorfismo con un método que reciba un Empleado e imprima su información. (2,5 pts)

RESPUESTA / SOLUCIÓN

```
class Empleado {
    private String nombre; private double salario;
    public Empleado(String nombre, double salario) { this.nombre=nombre; this.salario=salario; }
    public String toString() { return nombre + ", " + salario + " €"; }
}

class Directivo extends Empleado {
    private String departamento;
    public Directivo(String n, double s, String dep) { super(n, s); this.departamento=dep; }
    public String toString() { return super.toString() + " [" + departamento + "]"; }
}

class Tecnico extends Empleado {
    private String especialidad;
    public Tecnico(String n, double s, String esp) { super(n, s); this.especialidad=esp; }
    public String toString() { return super.toString() + " <" + especialidad + ">"; }
}

public class Main {
    static void mostrar(Empleado e) { System.out.println(e); }
    public static void main(String[] a) {
        mostrar(new Directivo("Ana", 3000, "IT"));
        mostrar(new Tecnico("Luis", 2000, "Redes"));
    }
}
```

Define una interfaz `Movable` con el método `void moverse()`. Define también una interfaz `Sonoro` con `void sonar()`. Crea una clase `Robot` que implemente ambas interfaces y una clase `Coche` que implemente solo Movable. Escribe un programa que cree objetos y los trate a través de las interfaces. (2,5 pts)

RESPUESTA / SOLUCIÓN

```
interface Movable { void moverse(); }
interface Sonoro { void sonar(); }
class Robot implements Movable, Sonoro {
    public void moverse() { System.out.println("El robot se mueve"); }
    public void sonar() { System.out.println("Bip bip"); }
}
```

```

class Coche implements Movable {
    public void moverse() { System.out.println("El coche avanza"); }
}

public class Main {
    public static void main(String[] a) {
        Movable[] m = { new Robot(), new Coche() };
        for (Movable x : m) x.moverse();
        ((Sonoro)m[0]).sonar();
    }
}

```

Crea una clase abstracta `Instrumento` con un método abstracto `tocar()`. Deriva al menos tres instrumentos (Piano, Guitarra, Bateria). Prepara un arreglo de Instrumento con varios objetos y haz que todos 'toquen' usando polimorfismo. Añade un contador estático de instrumentos creados. (2 pts)

RESPUESTA / SOLUCIÓN

```

abstract class Instrumento {
    static int total = 0;
    Instrumento() { total++; }
    abstract void tocar();
}

class Piano extends Instrumento { void tocar() { System.out.println("Piano: plin plin"); } }
class Guitarra extends Instrumento { void tocar() { System.out.println("Guitarra: strum"); } }
class Bateria extends Instrumento { void tocar() { System.out.println("Batería: bom bom"); } }

public class Main { public static void main(String[] a) {
    Instrumento[] orq = { new Piano(), new Guitarra(), new Bateria(), new Piano() };
    for (Instrumento i : orq) i.tocar();
    System.out.println("Total instrumentos: " + Instrumento.total);
}}

```

Parte 4: Pregunta teórica

1 punto

Explica la diferencia entre una clase abstracta y una interfaz en Java. Da un ejemplo de cuándo elegirías cada una.

RESPUESTA / SOLUCIÓN

Clase abstracta: puede tener atributos de instancia, constructores y métodos concretos. Solo se puede heredar de UNA. Ideal para compartir estado/implementación común entre clases muy relacionadas (p. ej., Figuras que comparten comportamiento).

Interfaz: declara un 'contrato' (métodos) que las clases implementan; desde Java 8 puede tener métodos default/static y constantes. Una clase puede implementar VARIAS. Ideal para definir capacidades ('puede volar', 'puede moverse') reutilizables entre clases no relacionadas.

Regla general: cuando hay una relación 'es-un' fuerte y código compartido → clase abstracta; cuando se quiere un contrato flexible o múltiples capacidades → interfaz.

Plantilla de respuestas

(Páginas en blanco)